HW1_alex_tiu

September 9, 2026

1 Problem B.5

[ ]:
```
# Loop to add all numbers between 1 and 99
total = 0

# Range is from inclusive to exclusive
for i in range(99 + 1):

    # If number is odd
    if (i % 2 != 0):
        total += i

print(total)
```

2 Problem B.6

[ ]:
```
# Problem B.6

# Use nested for loops to assign entires of a 5x5 matrix A

# make 5x5 matrix using lists
column = []

for i in range(1, 6):
    row = []
    for j in range(1, 6):
        row.append(i*j)

    column.append(row)

print(column)
```

[ ]:
```
# Using a matrix
import numpy as np

A = np.zeros((5,5))
```

```
for i in range(0, len(A)):
    for j in range(0, len(A)):
        A[i,j] = (i+1)*(j+1)

print(A)
```

3 Problem B.7

[ ]:
```
# Problem B.7

# Use a for loop to satisfy the recurrence relation of the Fibonacci sequence,␣
↪but do not use an array
# to compute and display the first 20 terms

# n + 1, don't include 0
first = 1
second = 1

for i in range(20):
    print(first)

    # add up first and second bc fib
    next = first + second
    first = second
    second = next
```

4 Problem B.8

[ ]:
```
# Problem B.8

# If bank account balance is less than 5000, the annual interest rate is 1%.
# If the account balance is betwee 5000 and 20,000 exclusive, the interest rate␣
↪is 2%
# Otherwise, it's 5%
accounts = [6000,2000,55000,100287,50]

def rates(x):
    if (x < 5000):
        print(f"1% rate: {x * 1.01}")
    elif (x < 20000):
        print(f"2% rate: {x * 1.02}")
    else:
        print(f"5% rate: {x * 1.05}")

for price in accounts:
```

```
    rates(price)
```

5 Problem B.9

[ ]:
```python
# Problem B.9

# Write a function that takes as input three variables and returns the largest␣
↪of the three.
# Do not use max()

# Step 1: begin function
def get_max(x, y, z):
    if (x >= y) and (x >= z):
        return x
    elif (y >= x) and (y >= z):
        return y
    else:
        return z

print(get_max(7,8,5))
print(get_max(70,10,1))
print(get_max(-1,20,25))
```

6 Problem B.10

[ ]:
```python
# Problem B.10

# Let the variable epsilon equal to 1. Use a while loop to keep dividing␣
↪epsilon by 2 until
# epsilon < 10^-6, and determine how many divisions were used.

# Create a counter for how many divisions are used
divisions = 0

# Initialize epsilon and tolerance
epsilon = 1
TOL = 10**-6

while (epsilon > TOL):
    epsilon /= 2
    divisions += 1

print(f"Num of divisions: {divisions}")
```

After n divisions, $\epsilon = 1/2^n$; since $2^{19} = 524288 < 10^6$ but $2^{20} = 1048576 > 10^6$, the smallest n for which $\epsilon < 10^{-6}$ is $n = 20$.

[ ]:
```python
# Problem B.12

import numpy as np

# Convert from radians to angles
def radtodeg(x):
    return (180 / np.pi) * x

print(radtodeg(np.pi / 6))
print(radtodeg(np.pi / 4))
print(radtodeg(np.pi / 3))
print(radtodeg(np.pi / 2))
print(radtodeg(np.pi))
```

[ ]:
```python
# Problem B.13

# Write a function which converts temperature between Celsius (C) and␣
↪Fahrenheit (F)

import numpy as np
import matplotlib.pyplot as plt
# Write function for celsius to fahrenheit
def convert_temp(temp, ctof=True):

    # if ctof is true, then it executes celsius to fahrenheit conversion
    # Default is that it is true
    if ctof:
        # takes celsius and multiplies by 1.8 and adds 32
        return ((9 / 5) * temp) + 32
    else:
        # basically flips the above equation to be in terms of fahrenheit␣
↪instead
        return (5 / 9) * (temp -32)



# Build the graphs
# Have a linspace

x = np.linspace(0, 100, 1000)
y = convert_temp(x)

plt.plot(x, y)
plt.xlabel("Celsius (°C)")
plt.ylabel("Fahrenheit (°F)")
plt.title("Fahrenheit vs. Celsius")
```

[ ]:
```python
# Build the graphs
# Have a linspace
x = np.linspace(0, 212, 1000)
y = convert_temp(x, False)

plt.plot(x, y)
plt.xlabel("Fahrenheit (°F)")
plt.ylabel("Celsius (°C)")
plt.title("Celsius vs. Fahrenheit")
```

[ ]:
```python
# Problem B.14

# Make a recursive function that evaluates factorials

def factorial(n):
    if n == 0 or n == 1:
        return 1

    return n * factorial(n - 1)

for i in range(11):
    print(f"{i}!: {factorial(i)}")
```